Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase VI · Diseño y construcción de optimizadores con IA

Módulo 24

Formular y resolver problemas de optimización

Programación lineal y entera, restricciones, convexidad, metaheurísticas y decisiones con predicciones

---

Resumen del módulo

Edición de octubre de 2026

Sobre este resumen

Este documento reúne lo indispensable de los apuntes del Módulo 24: el texto expositivo, todas las definiciones y todos los resultados (teoremas, proposiciones, lemas y corolarios), las tablas y las observaciones de contenido. Se han suprimido las demostraciones, los ejemplos, las lecturas, las figuras, los ejercicios y sus soluciones. La numeración de temas, secciones, enunciados y ecuaciones es la misma que en los apuntes, de modo que cualquier referencia puede buscarse allí; las que remiten a elementos suprimidos (un ejemplo, una figura, una ecuación de una demostración) conservan su número en los apuntes.

Notación

Símbolo	Significado
$x \in \mathbb{R}^n$, $\mathcal{X}$	vector de decisión y conjunto factible
f, g_i, h_j	función objetivo, restricciones de desigualdad ($g_i \le 0$) y de igualdad ($h_j = 0$)
θ, $\hat{\theta}$	parámetros (datos) del problema y su estimación o predicción
c, A, b	datos de un programa lineal $\max\{c^{\mathsf{T}}x : Ax \le b,\ x \ge 0\}$
y, y^*	variables duales; precios sombra en el óptimo
z^*, z_{LP}	valor óptimo de un problema y de su relajación lineal
M	constante de las restricciones big-M
$\mathcal{L}(x, \lambda, \nu)$	lagrangiana con multiplicadores $\lambda \ge 0$ (desigualdades) y ν (igualdades)
T	temperatura del recocido simulado
$\succ$	dominancia de Pareto ($u \succ v$: u domina a v)
ξ, Ξ	parámetro incierto y conjunto de incertidumbre
c_u, c_o	coste unitario por defecto y por exceso (vendedor de periódicos)
F, F^{-1}	función de distribución y función cuantil
hL, MW, MWh, h	hectolitros, megavatios, megavatios hora, horas

Índice

Sobre este resumen **ii**

1 Del problema real al modelo matemático **1**
1.1 Elementos de un modelo de optimización . . . 1
1.2 Anatomía de un optimizador con IA . . . 2
1.3 Mapa de métodos de la fase . . . 3

2 Programación lineal y entera mixta **5**
2.1 Programas lineales y vértices . . . 5
2.2 Dualidad y precios sombra . . . 5
2.3 Programación entera mixta . . . 6
2.4 Ramificación y acotación . . . 7

3 Problemas combinatorios y programación por restricciones **8**
3.1 Explosión combinatoria . . . 8
3.2 Rutas de vehículos con ventanas horarias . . . 8
3.3 Programación por restricciones y taller . . . 9

4 Optimización continua y convexa **11**
4.1 Problemas convexos . . . 11
4.2 Lagrangiana, dualidad y condiciones KKT . . . 11
4.3 Caso de estudio: despacho económico . . . 12

5 Metaheurísticas, optimización sin derivadas y multiobjetivo **14**
5.1 Optimización de caja negra . . . 14
5.2 Recocido simulado . . . 14
5.3 Algoritmos genéticos y estrategias evolutivas . . . 15
5.4 Optimización multiobjetivo . . . 15

6 Cuando la IA entra en la formulación: predecir y optimizar **17**
6.1 Predecir y después optimizar . . . 17
6.2 El vendedor de periódicos . . . 17
6.3 Programación estocástica y robusta . . . 18
6.4 Caso de estudio: plan de cerveza fresca con demanda predicha . . . 19

Bibliografía **20**

Tema 1

Del problema real al modelo matemático

Antes de elegir un algoritmo hay que saber qué se decide, qué se quiere mejorar y qué no se puede violar. Este tema fija el vocabulario de la modelización siguiendo el capítulo 1 de Bertsimas y Tsitsiklis [5], la clase 1 («Formulations») del curso 6.251J del MIT [29] y la clase 2 («Formulations of linear and non-linear programs») del curso 15.053 [30]; describe la arquitectura de un sistema que predice y decide, con dos aplicaciones industriales publicadas [15, 23], y traza el mapa de métodos que recorre la fase, con el capítulo 1 de Kochenderfer y Wheeler [27] como guía.

1.1 Elementos de un modelo de optimización

Un problema real llega como una descripción verbal: «queremos producir lo más rentable posible sin parar la línea y sin dejar a ningún cliente sin pedido». Modelizar consiste en separar lo que se elige de lo que se sabe y en escribir con precisión lo que se quiere y lo que se exige.

Definición 1.1 (Problema de optimización). Un *problema de optimización* es

$$\min_{x \in X} \; f(x) \qquad \text{s.a. } g_i(x) \le 0 \;\; (i = 1, \dots, m), \qquad h_j(x) = 0 \;\; (j = 1, \dots, p),$$

donde x es el vector de *variables de decisión*, X un conjunto básico (por ejemplo $\mathbb{R}^n$, $\mathbb{R}^n_{\ge 0}$ o $\mathbb{Z}^{n_1} \times \mathbb{R}^{n_2}$), f la *función objetivo* y g_i, h_j las *restricciones*. El *conjunto factible* es $\mathcal{X} = \{x \in X : g_i(x) \le 0,\ h_j(x) = 0\}$. Un punto $x^* \in \mathcal{X}$ es *óptimo (global)* si $f(x^*) \le f(x)$ para todo $x \in \mathcal{X}$, y *óptimo local* si la desigualdad se cumple en $\mathcal{X} \cap B(x^*, r)$ para algún $r > 0$. El problema es *infactible* si $\mathcal{X} = \varnothing$ y *no acotado* si $\inf_{\mathcal{X}} f = -\infty$.

Definición 1.2 (Datos, decisiones e indicadores). En un modelo se distinguen:

(a) los *parámetros* o datos θ (precios, capacidades, demandas, tiempos), que el decisor no controla y que pueden ser inciertos o predichos;

(b) las *variables de decisión* x, que el decisor fija;

(c) las *restricciones duras*, que ninguna decisión puede violar (seguridad, física, contratos), y las *restricciones blandas*, que pueden violarse con una penalización en el objetivo;

(d) el *objetivo* $f(x; \theta)$, un único escalar que se optimiza;

(e) los *indicadores clave* (KPIs, *key performance indicators*): magnitudes que se miden para vigilar el sistema (utilización, nivel de servicio, emisiones, tiempo de cálculo) y que no tienen por qué coincidir con el objetivo.

Observación 1.3. El objetivo es una decisión de diseño: si se optimiza el coste y se vigila el nivel de servicio, conviene comprobar que el óptimo del coste no hunde el servicio. Cuando varios KPIs son igualmente importantes, el problema es multiobjetivo (tema 5). Optimizar un indicador que solo aproxima lo que se quiere conduce a la ley de Goodhart que el módulo 21 estudió como sobreoptimización de la recompensa.

Muchos modelos que no parecen lineales lo son tras una reformulación. Las tres siguientes son las más útiles.

Proposición 1.4 (Reformulaciones elementales). *(i) Maximizar f equivale a minimizar $-f$:* $\max_{\mathcal{X}} f = -\min_{\mathcal{X}}(-f)$*, con los mismos puntos óptimos.*

(ii) Una igualdad $h(x) = 0$ equivale a las dos desigualdades $h(x) \le 0$ y $-h(x) \le 0$; una desigualdad $a^{\mathsf{T}}x \le b$ equivale a $a^{\mathsf{T}}x + s = b$ con una holgura *$s \ge 0$.*

(iii) Una variable libre $x_j \in \mathbb{R}$ puede sustituirse por $x_j = x_j^+ - x_j^-$ con $x_j^{\pm} \ge 0$.

Proposición 1.5 (Epígrafe de un máximo). *Sean $f_1, \dots, f_k$ funciones sobre $\mathcal{X}$. Los problemas*

$$(P)\ \min_{x\in\mathcal{X}} \max_i f_i(x) \qquad y \qquad (Q)\ \min_{x\in\mathcal{X},\, t\in\mathbb{R}} t \ \ \text{s.a. } f_i(x) \le t \ \ (i = 1, \dots, k)$$

tienen el mismo valor óptimo, y x^* es óptimo de (P) si y solo si $(x^*, \max_i f_i(x^*))$ es óptimo de (Q). Si las f_i son afines y $\mathcal{X}$ es un poliedro, (Q) es un programa lineal.

Proposición 1.6 (Suma de valores absolutos). *El problema* $\min_{x\in\mathcal{X}} \sum_{i=1}^{k} |a_i^{\mathsf{T}}x - b_i|$ *equivale a*

$$\min_{x\in\mathcal{X},\, e\in\mathbb{R}^k} \sum_{i=1}^{k} e_i \qquad \text{s.a. } -e_i \le a_i^{\mathsf{T}}x - b_i \le e_i \quad (i = 1, \dots, k).$$

1.2 Anatomía de un optimizador con IA

En la industria el modelo matemático es una pieza de un sistema mayor que se ejecuta de forma periódica (cada semana, cada hora, cada cinco minutos). La figura 1.2 muestra sus componentes.

Definición 1.8 (Optimizador con IA). Un *optimizador con IA* es un procedimiento que, a partir de observaciones w (contexto: fecha, temperatura, estado de la planta), produce una decisión

$$\hat{x}(w) \in \arg\min_{x\in\mathcal{X}(\hat{\theta})} f\big(x; \hat{\theta}\big), \qquad \hat{\theta} = m_\phi(w),$$

donde m_ϕ es un modelo aprendido de datos con parámetros ϕ que predice los parámetros inciertos del problema (o el propio valor de una decisión), y el conjunto factible incluye las restricciones de seguridad. La calidad del sistema se mide por el coste real $f(\hat{x}(w); \theta)$ con el θ verdadero, no por el error de predicción.

Los componentes de la figura 1.2 son:

1. *Datos*: históricos, tiempo real y datos maestros (capacidades, recetas, tarifas). Su calidad limita todo lo demás.
2. *Modelo predictivo*: estima demandas, tiempos, rendimientos o el estado futuro del sistema (regresión, árboles, redes; módulos 8–11).

3. *Modelo de optimización*: la formulación de la definición 1.1, con los parámetros predichos y las restricciones duras.
4. *Resolvedor*: el algoritmo elegido según la estructura (apartado 1.3).
5. *Decisión con salvaguardas*: comprobaciones independientes del modelo (límites físicos, cambios máximos entre periodos, plan de respaldo) antes de ejecutar.
6. *Monitorización*: KPIs, error de predicción, deriva de los datos, tiempo de cálculo y tasa de intervenciones humanas; disparan el reentrenamiento o la revisión del modelo.

1.3 Mapa de métodos de la fase

La estructura matemática del problema, no la moda, decide el método. El cuadro 1.2 resume la elección; la figura 1.3 la presenta como árbol de decisión.

Cuadro 1.2: Mapa de métodos: estructura del problema, familia y herramienta típica.

Estructura	Familia	Métodos y herramientas
objetivo y restricciones lineales, variables continuas	PL (LP)	símplex, punto interior; HiGHS (tema 2)
lineal con decisiones discretas (sí/no, lotes, turnos)	PLEM (MILP)	ramificación y acotación con cortes; HiGHS, SCIP, Gurobi (tema 2)
secuenciación, rutas, asignación con lógica compleja	combinatoria, programación por restricciones	CP-SAT, OR-Tools Routing, búsqueda local (tema 3)
convexo y diferenciable (cuadrático, cónico)	optimización convexa	punto interior; CVXPY con Clarabel u OSQP (tema 4)
no convexo y diferenciable	programación no lineal	SQP, punto interior, multiarranque; SciPy (tema 4)
caja negra barata, sin derivadas	metaheurísticas	recocido, genéticos, CMA-ES, NSGA-II (tema 5)
caja negra cara (simulación, experimento)	optimización bayesiana	modelo sustituto gaussiano y función de adquisición [17, 16]
decisiones secuenciales con incertidumbre	control y RL	control predictivo, aprendizaje por refuerzo (módulo 21) [38]
parámetros inciertos que se predicen	predecir y optimizar, estocástica, robusta	SAA, recurso, conjuntos de incertidumbre (tema 6)
espacio de programas o heurísticas	búsqueda con LLM y evaluador	FunSearch [35], AlphaEvolve [33], OPRO [40]

Observación 1.10 (Modelos de lenguaje en la optimización). Los LLM intervienen en la optimización de dos formas distintas. Como *asistentes de formulación*, traducen un enunciado en lenguaje natural a un modelo para un resolvedor clásico; OptiMUS [1] es un ejemplo publicado para PL y PLEM, y su salida debe verificarse como cualquier código. Como *generadores de candidatos* dentro de un bucle evolutivo con un evaluador automático, proponen programas o soluciones que el evaluador puntúa: FunSearch [35] encontró así construcciones matemáticas

y heurísticas de empaquetado, AlphaEvolve [33] extendió la idea a algoritmos completos y OPRO [40] usa el propio LLM como optimizador de instrucciones. En ambos casos la garantía procede del evaluador o del resolvedor, no del modelo de lenguaje; los módulos posteriores de la fase desarrollan este enfoque.

Cuadro 1.3: Tecnologías del tema 1: lenguajes de modelización y resolvedores generales.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
SciPy (`optimize`)	PL y PLEM con HiGHS, mínimos cuadrados, no lineal local	abierta (BSD-3)	prototipos y modelos pequeños sin dependencias extra
PuLP [34]	modelizar PL/PLEM y enviarlos a CBC, HiGHS, SCIP, Gurobi…	abierta (MIT)	modelos lineales legibles en Python
Pyomo	modelización algebraica general (lineal y no lineal)	abierta (BSD-3)	modelos grandes, con estructura por índices y varios resolvedores
OR-Tools [18]	CP-SAT, rutas, flujos, PL (GLOP)	abierta (Apache 2.0)	combinatoria, programación y rutas
CVXPY [12]	optimización convexa disciplinada	abierta (Apache 2.0)	problemas convexos con verificación automática
Gurobi, CPLEX	resolvedores comerciales de PL/PLEM/cuadráticos	comercial (licencia académica gratuita)	PLEM grandes en producción con exigencia de tiempo

Tema 2

Programación lineal y entera mixta

La programación lineal y la entera mixta resuelven la mayor parte de los problemas de planificación de la industria: producción, mezclas, transporte, turnos, inversiones. Este tema sigue los capítulos 2 a 5, 10 y 11 de Bertsimas y Tsitsiklis [5], que son las lecturas del curso 6.251J del MIT [29], y las clases 6 («Sensitivity analysis and shadow prices»), 10–12 (programación entera y ramificación y acotación) del curso 15.053 [30], con los capítulos 3 («Sensitivity Analysis») y 9 («Integer Programming») de Bradley, Hax y Magnanti [10]. La ramificación y acotación procede de Land y Doig [28]; el resolvedor de referencia es HiGHS [22, 24].

2.1 Programas lineales y vértices

Definición 2.1 (Programa lineal). Un *programa lineal* (PL) en forma canónica es

$$z^* = \max\ c^{\mathsf{T}} x \qquad \text{s.a. } Ax \le b, \quad x \ge 0,$$

con $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$ y $c \in \mathbb{R}^n$. Su conjunto factible $\mathcal{P} = \{x : Ax \le b,\ x \ge 0\}$ es un *poliedro*. Un punto $v \in \mathcal{P}$ es un *vértice* si no es combinación convexa $\lambda u + (1-\lambda)w$ de dos puntos distintos $u, w \in \mathcal{P}$ con $\lambda \in (0,1)$; equivalentemente, si en v hay n restricciones activas linealmente independientes [5].

Teorema 2.2 (Teorema fundamental de la programación lineal). *Si $\mathcal{P}$ es no vacío y acotado, el PL tiene un óptimo que es un vértice de $\mathcal{P}$.*

El *método símplex* (Dantzig) explota el teorema: recorre vértices adyacentes mejorando el objetivo hasta que ninguna arista mejora. Los *métodos de punto interior* avanzan por el interior del poliedro y tienen complejidad polinómica. HiGHS implementa ambos (su símplex dual revisado paralelo se describe en [24]) y un resolvedor de PLEM; SciPy lo usa en `linprog` y `milp` [22].

2.2 Dualidad y precios sombra

Definición 2.3 (Problema dual). El *dual* del PL de la definición 2.1 es

$$w^* = \min\ b^{\mathsf{T}} y \qquad \text{s.a. } A^{\mathsf{T}} y \ge c, \quad y \ge 0.$$

Cada variable dual y_i corresponde a la restricción i del primal.

Teorema 2.4 (Dualidad débil). *Si x es factible para el primal e y para el dual, entonces $c^{\mathsf{T}} x \le b^{\mathsf{T}} y$. En consecuencia $z^* \le w^*$, y si $c^{\mathsf{T}} x = b^{\mathsf{T}} y$ ambos son óptimos.*

Teorema 2.5 (Dualidad fuerte). *Si el primal tiene óptimo, también lo tiene el dual y $z^* = w^*$ (Bertsimas y Tsitsiklis [5], cap. 4).*

Corolario 2.6 (Holgura complementaria). *Sean x e y factibles. Ambos son óptimos si y solo si*

$$y_i\,\big(b_i - (Ax)_i\big) = 0 \quad \text{para todo } i \qquad y \qquad x_j\,\big((A^{\mathsf{T}}y)_j - c_j\big) = 0 \quad \text{para todo } j.$$

La holgura complementaria tiene una lectura económica directa: un recurso que sobra (restricción no activa) tiene precio cero, y un producto que no se fabrica no cubre con su margen el valor de los recursos que consumiría.

Proposición 2.7 (Precios sombra). *Sea $z^*(b)$ el valor óptimo en función del término independiente. La función z^* es cóncava en el conjunto de b con primal factible y acotado, y si y^* es una solución dual óptima para b, para todo b' de ese conjunto*

$$z^*(b') \le z^*(b) + (y^*)^{\mathsf{T}}(b' - b).$$

Si el dual óptimo es único, z^* es diferenciable en b con $\partial z^*/\partial b_i = y_i^*$: el precio sombra y_i^* *es lo que aumenta el valor óptimo por unidad adicional del recurso i, mientras la base óptima no cambia.*

2.3 Programación entera mixta

Definición 2.9 (PLEM y relajación lineal). Un *programa lineal entero mixto* (PLEM, en inglés MILP) es un PL en el que un subconjunto J de variables debe ser entero: $\max\{c^{\mathsf{T}}x : Ax \le b,\ x \ge 0,\ x_j \in \mathbb{Z}\ (j \in J)\}$. Su *relajación lineal* es el mismo problema sin la condición de integralidad; su valor se denota z_{LP}.

Proposición 2.10 (La relajación acota). *En un problema de maximización, $z^* \le z_{\mathrm{LP}}$; en uno de minimización, $z^* \ge z_{\mathrm{LP}}$. Si la solución de la relajación cumple la integralidad, es óptima para el PLEM.*

La diferencia relativa entre el mejor valor entero conocido (el *incumbente*) y la mejor cota se llama *gap*; los resolvedores se detienen cuando es menor que una tolerancia (por defecto, del orden de 10^{-4}).

Las variables binarias $y \in \{0,1\}$ permiten modelizar decisiones lógicas con restricciones lineales:

(a) *coste fijo o de preparación*: $x \le My$ con coste fy en el objetivo; si se produce ($x > 0$), se paga f;

(b) *lote mínimo*: $Ly \le x \le My$; o no se produce, o se producen al menos L unidades;

(c) *implicación* «si se elige A, se elige B»: $y_A \le y_B$; *exclusión*: $y_A + y_B \le 1$; *al menos k de n*: $\sum_i y_i \ge k$;

(d) *disyunción* «se cumple $a^{\mathsf{T}}x \le b$ o $d^{\mathsf{T}}x \le e$»: $a^{\mathsf{T}}x \le b + M\delta$, $d^{\mathsf{T}}x \le e + M(1-\delta)$, $\delta \in \{0,1\}$.

Proposición 2.11 (Validez y fuerza de una cota big-M). *Sea $\mathcal{X}$ el conjunto factible sin la restricción $x \le My$ y sea $U = \sup\{x : x \in \mathcal{X}\}$.*

(i) La restricción $x \le My$, con $y \in \{0,1\}$, no elimina ninguna solución con $y = 1$ si y solo si $M \ge U$.

(ii) Si $U \le M \le M'$, la relajación lineal con M está contenida en la relajación con M'; la menor cota válida, $M = U$, da la relajación más ajustada.

Con una M grande la relajación permite valores de y diminutos, por ejemplo $y = x/M$, y paga solo una fracción del coste fijo: la cota es mala y el árbol de búsqueda crece. Una M desmesurada causa además errores numéricos, porque los resolvedores trabajan con tolerancias del orden de 10^{-6} a 10^{-9}.

2.4 Ramificación y acotación

La *ramificación y acotación* (*branch and bound*) [28] explora un árbol de subproblemas. En cada nodo se resuelve la relajación lineal con las cotas acumuladas. Para un problema de maximización:

1. si la relajación es infactible, el nodo se *poda*;
2. si su valor no supera el del incumbente, se poda *por cota*: ningún descendiente puede mejorar (proposición 2.10);
3. si la solución es entera, se actualiza el incumbente y se poda;
4. si no, se elige una variable x_j con valor fraccionario x_j^* y se crean dos hijos con $x_j \le \lfloor x_j^* \rfloor$ y $x_j \ge \lceil x_j^* \rceil$.

Si todas las variables enteras están acotadas, el árbol es finito y el incumbente final es óptimo: cada nodo podado lo es con una justificación válida y las ramas cubren todos los enteros factibles. Si los coeficientes del objetivo son enteros, el valor de cualquier solución entera también lo es, y un nodo puede podarse cuando $\lfloor z_{\mathrm{LP}} \rfloor$ no supera al incumbente. Los resolvedores modernos añaden preproceso, planos de corte (desigualdades válidas que eliminan soluciones fraccionarias; clase 13 de [30]), heurísticas primales y reglas de ramificación elaboradas.

Cuadro 2.1: Tecnologías del tema 2: programación lineal y entera mixta.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
HiGHS [22]	PL (símplex dual, punto interior), PLEM, cuadrático	abierta (MIT)	resolvedor abierto de referencia; incluido en SciPy
SciPy `linprog/milp`	interfaz matricial a HiGHS, con duales	abierta (BSD-3)	modelos pequeños y docencia
PuLP [34]	modelización por nombres; CBC, HiGHS, SCIP…	abierta (MIT)	modelos de planificación legibles
SCIP	PLEM y no lineal entero, con cortes y ramificación configurables	abierta (Apache 2.0 desde la 8.0.3)	investigación y PLEM difíciles sin licencia comercial
Gurobi, CPLEX	PL/PLEM a gran escala	comercial	producción con exigencias de tiempo y soporte
OR-Tools (GLOP, MathOpt)	PL y envoltorio de resolvedores	abierta (Apache 2.0)	si ya se usa OR-Tools para rutas o CP

Tema 3

Problemas combinatorios y programación por restricciones

Rutas, secuencias y calendarios tienen un número de soluciones que crece de forma factorial con el tamaño. Este tema estudia cómo formularlos y qué herramientas los resuelven en la práctica. Sigue la clase 17 («Traveling salesman problem») del curso 15.053 del MIT [30] para la complejidad y las heurísticas, el capítulo 10 de Bertsimas y Tsitsiklis [5] para las formulaciones enteras, el artículo de Solomon [37] para las rutas con ventanas horarias, el de Johnson [25] para el taller con dos máquinas y la documentación de OR-Tools [20, 19, 18] para la programación por restricciones.

3.1 Explosión combinatoria

Proposición 3.1 (Número de circuitos). *Con $n \geq 3$ ciudades y distancias simétricas hay $(n-1)!/2$ circuitos hamiltonianos distintos.*

Con $n = 20$ hay $19!/2 \approx 6{,}1 \cdot 10^{16}$ circuitos. El problema del viajante (TSP) es NP-difícil: no se conoce un algoritmo polinómico, y encontrarlo daría uno para todos los problemas NP-completos [30]. En la práctica se combinan tres enfoques:

(a) *métodos exactos*: PLEM con ramificación, cortes y generación de columnas, o programación dinámica para instancias pequeñas; dan el óptimo y una cota;

(b) *heurísticas constructivas*: vecino más próximo, inserción, ahorros; dan una solución rápida;

(c) *búsqueda local y metaheurísticas*: intercambios 2-opt y 3-opt, recolocación de clientes, búsqueda local guiada, recocido (tema 5).

Definición 3.2 (Movimiento 2-opt). Dado un circuito $(\dots, a, b, \dots, c, d, \dots)$, el movimiento 2-opt elimina las aristas (a, b) y (c, d) y añade (a, c) y (b, d), invirtiendo el tramo de b a c. La longitud cambia en $\Delta = d(a,c) + d(b,d) - d(a,b) - d(c,d)$; el movimiento se aplica si $\Delta < 0$. Un circuito es *2-óptimo* si ningún movimiento 2-opt lo mejora.

En el plano euclídeo, un circuito con dos aristas que se cruzan nunca es 2-óptimo: deshacer el cruce lo acorta, por la desigualdad triangular aplicada a los dos triángulos que forman las diagonales (ejercicio 3.3).

3.2 Rutas de vehículos con ventanas horarias

Definición 3.3 (VRPTW). En el *problema de rutas de vehículos con ventanas horarias* (VRPTW) se tiene un depósito 0, clientes $1, \dots, n$ con demanda q_i, tiempo de servicio s_i y

ventana $[a_i, b_i]$, una flota de K vehículos de capacidad Q y tiempos de viaje t_{ij}. Se buscan rutas que empiecen y acaben en el depósito, visiten cada cliente exactamente una vez, no superen la capacidad y empiecen cada servicio dentro de su ventana (se puede esperar si se llega antes de a_i), minimizando el tiempo total de viaje.

Una formulación PLEM usa $x_{ij} \in \{0, 1\}$ (un vehículo va de i a j), el instante de inicio de servicio T_i y la carga acumulada u_i:

$$\begin{aligned}
\min\ & \sum_{i \neq j} t_{ij} x_{ij} \\
\text{s.a.}\ & \sum_j x_{ij} = 1, \quad \sum_j x_{ji} = 1 \quad (i = 1, \dots, n), \qquad \sum_j x_{0j} \leq K, \\
& T_j \geq T_i + s_i + t_{ij} - M_{ij}(1 - x_{ij}) \quad (i \neq j,\ j \geq 1), \qquad a_i \leq T_i \leq b_i, \\
& u_j \geq u_i + q_j - Q(1 - x_{ij}) \quad (i \neq j,\ j \geq 1), \qquad q_i \leq u_i \leq Q.
\end{aligned}$$

Las restricciones de tiempo y de carga eliminan además los subcircuitos, porque a lo largo de un ciclo sin depósito T debería crecer indefinidamente.

Proposición 3.4 (Cota big-M de las ventanas). *La restricción temporal con* $M_{ij} = \max(0,\ b_i + s_i + t_{ij} - a_j)$ *es válida: si* $x_{ij} = 1$ *impone la llegada correcta y si* $x_{ij} = 0$ *no elimina ninguna combinación de instantes que respete las ventanas.*

Con más de unas decenas de clientes, la formulación anterior es lenta, y los resolvedores de rutas usan búsqueda local sobre las propias rutas. OR-Tools modela el tiempo como una *dimensión*: una variable acumulada por nodo, con una holgura que representa la espera y un rango que impone la ventana [20].

3.3 Programación por restricciones y taller

Definición 3.6 (Problema de satisfacción de restricciones). Un *problema de satisfacción de restricciones* consta de variables $v_1, \dots, v_k$ con dominios finitos $D_1, \dots, D_k$ y un conjunto de restricciones, cada una una relación sobre algunas variables. La *programación por restricciones* (CP) lo resuelve alternando *propagación* (eliminar de los dominios valores que no pueden formar parte de ninguna solución) y *búsqueda* (fijar una variable y retroceder si se llega a un dominio vacío). Con un objetivo, se añade la restricción «objetivo mejor que el incumbente» cada vez que se encuentra una solución.

La CP brilla cuando las restricciones son lógicas o de secuenciación, difíciles de expresar linealmente. Sus restricciones globales encapsulan estructuras frecuentes con propagadores eficientes: `AllDifferent`, `NoOverlap` (intervalos que no se solapan en un recurso), `Cumulative` (recurso con capacidad). CP-SAT, de OR-Tools, combina propagación de restricciones con técnicas de resolvedores SAT y de programación entera, y exige coeficientes enteros [18].

Definición 3.7 (Job-shop). En el problema de *taller* (*job-shop*) hay J trabajos; el trabajo j es una secuencia de operaciones $(m_{j1}, p_{j1}), \dots, (m_{jk_j}, p_{jk_j})$ que deben ejecutarse en ese orden, cada una en una máquina m y con duración p. Cada máquina procesa una operación a la vez y sin interrupción. Se minimiza el *makespan* $C_{\max}$, el instante en que termina el último trabajo.

En CP-SAT cada operación es un intervalo $[S_{jk}, S_{jk} + p_{jk})$; se impone $S_{j,k+1} \geq S_{jk} + p_{jk}$, una restricción `NoOverlap` por máquina y $C_{\max} \geq S_{jk_j} + p_{jk_j}$ [19]. En PLEM, cada par de operaciones en la misma máquina necesita una binaria de orden y dos restricciones big-M, y la relajación lineal resultante es muy débil.

Proposición 3.8 (Cotas inferiores del makespan). *Para todo programa factible,*

$$C_{\max} \;\geq\; \max\Big(\max_{m} \sum_{(j,k)\colon m_{jk}=m} p_{jk},\quad \max_{j} \sum_{k} p_{jk}\Big).$$

Cuadro 3.1: Tecnologías del tema 3: combinatoria, rutas y programación por restricciones.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
OR-Tools CP-SAT [18]	CP con intervalos, `NoOverlap`, `Cumulative`; también PLEM con coeficientes enteros	abierta (Apache 2.0)	programación de talleres, turnos y calendarios
OR-Tools Routing [20]	VRP con capacidades, ventanas, recogidas y entregas	abierta (Apache 2.0)	rutas reales de decenas a miles de paradas
PLEM (HiGHS, SCIP, Gurobi)	formulaciones con binarias de orden o de arco	abierta o comercial	instancias pequeñas, o con fuerte estructura lineal
Programación dinámica propia	exacto sobre subconjuntos	código propio	certificar instancias pequeñas
Heurísticas propias (2-opt, inserción)	solución rápida y controlable	código propio	tiempo real o reglas de negocio muy específicas

Tema 4

Optimización continua y convexa

Cuando las variables son continuas y el problema es convexo, todo mínimo local es global, existen certificados de optimalidad y los multiplicadores tienen interpretación económica. Este tema sigue los capítulos 4 («Convex optimization problems») y 5 («Duality»), en especial las secciones 5.5 («Optimality conditions») y 5.6 («Perturbation and sensitivity analysis»), de Boyd y Vandenberghe [8], texto del curso EE364a de Stanford; el capítulo 12 de Nocedal y Wright [32] para las condiciones de Karush–Kuhn–Tucker en problemas no convexos; y el capítulo 3 («Economic Dispatch of Thermal Units and Methods of Solution») de Wood, Wollenberg y Sheblé [39] para el caso de estudio. Las herramientas son CVXPY [12, 13] y SciPy. El módulo 7 (tema 1) introdujo la convexidad en el contexto del entrenamiento; aquí se añaden las restricciones.

4.1 Problemas convexos

Definición 4.1 (Problema convexo). El problema de la definición 1.1 es *convexo* si $X = \mathbb{R}^n$, f y las g_i son funciones convexas y las h_j son afines, $h_j(x) = a_j^{\mathsf{T}} x - b_j$. Su conjunto factible es entonces convexo, como intersección de subniveles de funciones convexas e hiperplanos.

Teorema 4.2 (Local implica global). *En un problema convexo, todo óptimo local es óptimo global.*

Las clases convexas más usadas, de menor a mayor generalidad, son la programación lineal, la cuadrática (objetivo $\tfrac{1}{2} x^{\mathsf{T}} P x + q^{\mathsf{T}} x$ con $P \succeq 0$ y restricciones lineales), la cónica de segundo orden y la semidefinida. CVXPY sigue las reglas de la *programación convexa disciplinada* (DCP): construye las expresiones con funciones atómicas de curvatura conocida y reglas de composición que garantizan la convexidad, de modo que rechaza un modelo cuya convexidad no puede certificar, y lo traduce a la forma cónica que aceptan resolvedores como Clarabel, SCS u OSQP [12, 13].

4.2 Lagrangiana, dualidad y condiciones KKT

Definición 4.3 (Lagrangiana y función dual). Para el problema de la definición 1.1 con $X = \mathbb{R}^n$, la *lagrangiana* es

$$\mathcal{L}(x, \lambda, \nu) = f(x) + \sum_{i=1}^{m} \lambda_i g_i(x) + \sum_{j=1}^{p} \nu_j h_j(x), \qquad \lambda \geq 0,$$

y la *función dual* es $q(\lambda, \nu) = \inf_x \mathcal{L}(x, \lambda, \nu)$.

Proposición 4.4 (Dualidad débil). *Si $\lambda \geq 0$, entonces $q(\lambda, \nu) \leq f(x)$ para todo x factible; por tanto $q(\lambda, \nu) \leq p^\ast$, el valor óptimo.*

En un problema convexo que cumple la *condición de Slater* (existe un punto estrictamente factible, $g_i(x) < 0$ para las restricciones no afines), la dualidad es fuerte: $\max q = p^*$ (Boyd y Vandenberghe [8], cap. 5).

Teorema 4.5 (Condiciones de Karush–Kuhn–Tucker). *Sean f, g_i y h_j diferenciables. Se dice que (x^*, λ^*, ν^*) cumple las condiciones KKT si*

(i) factibilidad primal*: $g_i(x^*) \le 0$, $h_j(x^*) = 0$;*

(ii) factibilidad dual*: $\lambda^* \ge 0$;*

(iii) holgura complementaria*: $\lambda_i^* g_i(x^*) = 0$ para todo i;*

(iv) estacionariedad*: $\nabla f(x^*) + \sum_i \lambda_i^* \nabla g_i(x^*) + \sum_j \nu_j^* \nabla h_j(x^*) = 0$.*

Si el problema es convexo y se cumplen las condiciones KKT, x^* es óptimo global. Si el problema es convexo y cumple la condición de Slater, todo óptimo satisface las KKT para algún (λ^*, ν^*) [8]. En problemas no convexos, las KKT son necesarias en un mínimo local bajo una cualificación de restricciones, pero no suficientes [32].

Proposición 4.6 (Multiplicadores como sensibilidades). *Sea el problema convexo perturbado $\min f(x)$ s.a. $g_i(x) \le u_i$, $h_j(x) = w_j$, con valor óptimo $p^*(u, w)$, y sea (λ^*, ν^*) dual óptimo del problema sin perturbar con dualidad fuerte. Entonces*

$$p^*(u, w) \;\ge\; p^*(0, 0) - \lambda^{*\mathsf{T}} u - \nu^{*\mathsf{T}} w,$$

y si p^* es diferenciable en $(0, 0)$, $\partial p^*/\partial u_i = -\lambda_i^*$ y $\partial p^*/\partial w_j = -\nu_j^*$.

Es la generalización de los precios sombra de la proposición 2.7: relajar una restricción activa abarata el óptimo, en primera aproximación, en su multiplicador por unidad.

4.3 Caso de estudio: despacho económico

Observación 4.8 (Cuando se pierde la convexidad). El caso anterior deja de ser convexo con detalles realistas: costes con efecto de apertura de válvulas (términos sinusoidales), mínimos técnicos con arranque y parada (variables binarias, el problema de asignación de unidades del cap. 4 de [39]) o mezclas con calidades que se multiplican por caudales (términos bilineales). Entonces las KKT son solo necesarias, los métodos locales (SQP, punto interior) pueden quedarse en óptimos locales y conviene arrancar desde varios puntos, usar un resolvedor global o reformular. En el ajuste de redes neuronales del módulo 7 el problema tampoco es convexo, pero allí basta con un buen mínimo local.

Cuadro 4.1: Tecnologías del tema 4: optimización continua.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
CVXPY [12]	modelización convexa con verificación DCP y duales	abierta (Apache 2.0)	cualquier problema convexo de tamaño medio
Clarabel, SCS, OSQP	resolvedores cónicos (Clarabel, SCS) y cuadráticos (OSQP)	abiertos (Apache 2.0; SCS, MIT)	los usa CVXPY; OSQP, para control predictivo rápido
SciPy `minimize`	SLSQP, `trust-constr`, L-BFGS-B	abierta (BSD-3)	no lineal local con pocas variables
Ipopt	punto interior no lineal a gran escala	abierta (EPL-2.0)	no lineal grande con derivadas (vía Pyomo o CasADi)
Gurobi	cuadrático y no convexo bilineal global	comercial	mezclas y redes con términos bilineales

Tema 5

Metaheurísticas, optimización sin derivadas y multiobjetivo

Cuando el objetivo solo puede evaluarse (un simulador, un código heredado, un ensayo) o cuando la estructura del problema es tan irregular que ningún resolvedor exacto escala, se recurre a métodos que solo usan evaluaciones de f. Este tema sigue los capítulos 7 («Direct Methods»), 8 («Stochastic Methods»), 9 («Population Methods») y 12 («Multiobjective Optimization») de Kochenderfer y Wheeler [27], la clase 17 del curso 15.053 [30] para el recocido, y los artículos originales de Kirkpatrick, Gelatt y Vecchi [26] (recocido simulado), Hansen [21] (CMA-ES) y Deb *et al.* [11] (NSGA-II).

5.1 Optimización de caja negra

Definición 5.1 (Problema de caja negra). Un problema $\min_{x\in\mathcal{X}} f(x)$ es de *caja negra* si solo se dispone de un oráculo que devuelve $f(x)$ (quizá con ruido) para cada x consultado, sin gradientes ni estructura explícita. Su coste se mide por el *número de evaluaciones*.

Hay dos regímenes. Si cada evaluación es barata (milisegundos), se pueden permitir miles o millones de evaluaciones y se usan metaheurísticas. Si es cara (horas de simulación o un experimento de planta), cada evaluación debe elegirse con cuidado con un modelo sustituto, como en la optimización bayesiana [17, 16], que la fase desarrolla más adelante. Los métodos directos locales, como Nelder–Mead o Powell (cap. 7 de [27]), son adecuados cuando el problema es continuo, de pocas variables y con un único valle.

5.2 Recocido simulado

El recocido simulado [26] imita el enfriamiento lento de un metal. Desde una solución x se propone un vecino x' (por ejemplo, intercambiar dos departamentos) con un mecanismo simétrico, $\mathbb{P}(x \to x') = \mathbb{P}(x' \to x)$, y se acepta con la probabilidad de Metropolis

$$\alpha(x, x') = \min\Big(1,\ e^{-(f(x')-f(x))/T}\Big),$$

donde la temperatura $T > 0$ disminuye según un calendario, típicamente $T_{k+1} = \rho T_k$ con $\rho \in (0{,}9;\ 0{,}999)$. Con T alta se aceptan muchos empeoramientos y se explora; con T baja solo se aceptan mejoras y se explota.

Proposición 5.2 (Distribución estacionaria de Boltzmann). *Sea $\mathcal{X}$ finito, la propuesta simétrica e irreducible y T fija. La cadena de Markov del recocido satisface el equilibrio detallado con*

$$\pi_T(x) = \frac{e^{-f(x)/T}}{\sum_{z\in\mathcal{X}} e^{-f(z)/T}},$$

que es por tanto una distribución estacionaria. Cuando $T \to 0$, π_T se concentra uniformemente en el conjunto de mínimos globales.

La distribución de Boltzmann es la del módulo 6 (tema 5). La garantía de convergencia al óptimo exige calendarios logarítmicos, demasiado lentos; en la práctica se usa el geométrico y varias ejecuciones independientes.

5.3 Algoritmos genéticos y estrategias evolutivas

Un *algoritmo genético* mantiene una población de soluciones y la renueva con tres operadores (cap. 9 de [27]): *selección* de padres favoreciendo a los mejores (por ejemplo, por torneo entre dos individuos al azar), *cruce* que combina dos padres y *mutación* que perturba un hijo. Con *elitismo* los mejores pasan intactos a la generación siguiente. La representación debe respetar las restricciones: para permutaciones, el *cruce por orden* (OX) copia un tramo del primer padre y completa los huecos con los genes que faltan en el orden en que aparecen en el segundo, de modo que el hijo sigue siendo una permutación.

Definición 5.3 (CMA-ES). La *estrategia evolutiva con adaptación de la matriz de covarianza* (CMA-ES) [21] optimiza en $\mathbb{R}^n$ manteniendo una distribución $\mathcal{N}(m, \sigma^2 C)$. En cada generación:

1. muestrea λ candidatos $x_k = m + \sigma y_k$, con $y_k \sim \mathcal{N}(0, C)$, y los ordena por f;
2. actualiza la media con los μ mejores: $m \leftarrow \sum_{i=1}^{\mu} w_i x_{i:\lambda}$, con pesos positivos decrecientes que suman 1; la referencia propone $w_i \propto \ln \frac{\lambda+1}{2} - \ln i$;
3. actualiza C con los pasos que tuvieron éxito (actualizaciones de rango uno y de rango μ) y ajusta σ comparando la longitud del camino evolutivo con la esperada bajo selección aleatoria.

El *tamaño efectivo de selección* es $\mu_{\text{eff}} = 1/\sum_i w_i^2$.

CMA-ES aprende la forma del valle: en un problema mal condicionado, C tiende a una matriz proporcional a la inversa del hessiano, de modo que se comporta como un método de segundo orden sin derivadas. Es invariante a transformaciones monótonas de f (solo usa el orden) y a rotaciones del espacio. Es una elección sólida para calibrar simuladores de caja negra con decenas de parámetros continuos y miles de evaluaciones posibles.

5.4 Optimización multiobjetivo

Definición 5.4 (Dominancia y frente de Pareto). Sea $F = (f_1, \ldots, f_k)$ un vector de objetivos que se minimizan. Se dice que x *domina* a x', $x \succ x'$, si $f_i(x) \le f_i(x')$ para todo i y $f_j(x) < f_j(x')$ para algún j. Un punto factible es *Pareto-óptimo* si ningún otro punto factible lo domina. La imagen $F(\mathcal{X}^{\mathrm{P}})$ del conjunto de Pareto-óptimos es el *frente de Pareto*.

Proposición 5.5 (Suma ponderada). *Si $w_i > 0$ para todo i y x^* minimiza $\sum_i w_i f_i(x)$ en $\mathcal{X}$, entonces x^* es Pareto-óptimo. El recíproco es falso: los puntos del frente que no están en la frontera de su envolvente convexa no minimizan ninguna suma ponderada.*

El método de la ε-restricción, $\min f_1$ s.a. $f_2 \le \varepsilon$, sí alcanza los puntos no apoyados variando

ε (ejercicio 5.7). Para problemas de caja negra, el *NSGA-II* [11] combina un algoritmo genético con dos ideas:

(a) *ordenación no dominada*: la población se divide en frentes $\mathcal{F}_1$ (no dominados), $\mathcal{F}_2$ (no dominados al quitar $\mathcal{F}_1$), etc., y el rango de un individuo es el índice de su frente;

(b) *distancia de aglomeración* (*crowding distance*): dentro de un frente, para cada objetivo se ordenan los puntos, los extremos reciben distancia infinita y cada punto interior suma $(f_k^{\text{sig}} - f_k^{\text{ant}})/(f_k^{\max} - f_k^{\min})$; se prefieren los puntos aislados para mantener la diversidad.

La selección por torneo compara primero el rango y después la distancia, y la nueva generación se forma con los mejores frentes de la unión de padres e hijos (elitismo).

Cuadro 5.1: Tecnologías del tema 5: metaheurísticas y multiobjetivo.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
`cma` (Hansen *et al.*)	CMA-ES de referencia, con restricciones de caja y ruido	abierta (BSD-3)	calibrar simuladores continuos de caja negra
pymoo [7]	NSGA-II, NSGA-III, indicadores de calidad de frentes	abierta (Apache 2.0)	problemas multiobjetivo con operadores estándar
SciPy `optimize`	recocido dual, evolución diferencial, Nelder–Mead, Powell	abierta (BSD-3)	continuo, pocas variables, sin dependencias extra
Optuna	búsqueda de hiperparámetros (muestreadores TPE y CMA-ES, multiobjetivo)	abierta (MIT)	ajustar modelos o configuraciones con evaluaciones caras
Código propio	recocido, búsqueda tabú, genéticos sobre una representación específica	propio	combinatoria con vecindarios propios del dominio

Tema 6

Cuando la IA entra en la formulación: predecir y optimizar

En los temas anteriores los datos eran conocidos. En la práctica la demanda, los tiempos de viaje o los rendimientos se *predicen*, y la decisión hereda el error de la predicción. Este tema estudia cómo combinar ambos pasos. Sigue el capítulo 1 («Introduction and Examples») de Birge y Louveaux [6] y las transparencias de programación estocástica del curso EE364a de Stanford [9]; el capítulo 17 («Optimization under Uncertainty») de Kochenderfer y Wheeler [27]; los artículos de Elmachtoub y Grigas [14] y de Bertsimas y Kallus [3] sobre la integración de predicción y decisión; el de Bertsimas y Sim [4] y el libro de Ben-Tal, El Ghaoui y Nemirovski [2] para la optimización robusta, y el de Smith y Winkler [36] para la maldición del optimizador.

6.1 Predecir y después optimizar

Definición 6.1 (Predecir y después optimizar). Sea un problema $\min_{x\in\mathcal{X}} \mathbb{E}[c(x,\xi) \mid w]$, donde ξ es un parámetro incierto (la demanda) y w el contexto observado (temperatura, festivos). El enfoque *predecir y después optimizar* entrena un modelo $\hat{\xi} = m_\phi(w)$ y resuelve $\hat{x}(w) \in \arg\min_{x\in\mathcal{X}} c(x,\hat{\xi})$, sustituyendo el parámetro por su predicción puntual.

Proposición 6.2 (Cuándo basta la media). *Si el coste es lineal en el parámetro, $c(x,\xi) = \xi^{\mathsf{T}}x$, y $\mathcal{X}$ no depende de ξ, la decisión que minimiza el coste esperado condicionado es la que resulta de sustituir ξ por su media condicionada $\mathbb{E}[\xi \mid w]$.*

La proposición explica el éxito del enfoque cuando la incertidumbre está en los costes de un PL (rutas con tiempos inciertos y objetivo lineal) y su fracaso cuando el coste es no lineal en ξ: con inventario, el coste de quedarse corto y el de sobrar son distintos, y la media deja de ser suficiente. Incluso en el caso lineal, dos modelos con el mismo error cuadrático pueden inducir decisiones de calidad muy distinta: lo que importa no es el error en la predicción, sino el error en la decisión, que es lo que mide la pérdida SPO (*smart predict-then-optimize*) de Elmachtoub y Grigas [14] (ejercicio 6.3). Bertsimas y Kallus [3] proponen, en lugar de predecir un valor, aprender pesos sobre escenarios históricos parecidos al contexto y optimizar contra esa distribución empírica.

6.2 El vendedor de periódicos

Definición 6.3 (Problema del vendedor de periódicos). Se decide una cantidad $q \geq 0$ antes de conocer la demanda D, con función de distribución F. Cada unidad que falta cuesta $c_u > 0$ (margen perdido) y cada unidad que sobra, $c_o > 0$ (coste menos valor de saldo). El

coste esperado es

$$G(q) = \mathbb{E}\big[c_u(D-q)^+ + c_o(q-D)^+\big].$$

Teorema 6.4 (Fractil crítico). *Si F es continua, $q^* = F^{-1}\big(c_u/(c_u+c_o)\big)$ minimiza G. Si D es discreta, es óptimo el menor q con $F(q) \geq c_u/(c_u+c_o)$.*

Si la demanda es normal, $q^* = \mu + \sigma\,\Phi^{-1}(c_u/(c_u+c_o))$: se produce por encima de la media cuando el margen perdido supera al coste del sobrante. Predecir la media, por precisa que sea, no basta; hace falta un *cuantil*, y por tanto una estimación de la incertidumbre calibrada fuera de muestra.

6.3 Programación estocástica y robusta

Definición 6.5 (Programa estocástico en dos etapas). En un *programa en dos etapas* se fija una decisión x antes de observar ξ y, una vez observado, una decisión de recurso y:

$$\mathrm{RP} = \min_{x\in\mathcal{X}} \; c^{\mathsf{T}}x + \mathbb{E}_{\xi}\big[Q(x,\xi)\big], \qquad Q(x,\xi) = \min_{y\in\mathcal{Y}(x,\xi)} q(\xi)^{\mathsf{T}}y.$$

La *aproximación por media muestral* (SAA) sustituye la esperanza por la media sobre S escenarios $\xi^1,\dots,\xi^S$ y, si todo es lineal, da un único PL con una copia de y por escenario [9].

Definición 6.6 (Valor de la solución estocástica y de la información perfecta). Para un problema de minimización, sea $\bar{x}$ la solución del *problema del valor esperado* (sustituir ξ por $\mathbb{E}\xi$), $\mathrm{EEV} = c^{\mathsf{T}}\bar{x} + \mathbb{E}[Q(\bar{x},\xi)]$ su coste esperado real, y $\mathrm{WS} = \mathbb{E}_{\xi}[\min_x (c^{\mathsf{T}}x + Q(x,\xi))]$ el coste esperado si se conociera ξ antes de decidir (*wait-and-see*). Se definen el *valor de la solución estocástica* $\mathrm{VSS} = \mathrm{EEV} - \mathrm{RP}$ y el *valor esperado de la información perfecta* $\mathrm{EVPI} = \mathrm{RP} - \mathrm{WS}$ [6].

Proposición 6.7. $\mathrm{WS} \leq \mathrm{RP} \leq \mathrm{EEV}$*; en consecuencia,* $\mathrm{VSS} \geq 0$ *y* $\mathrm{EVPI} \geq 0$.

La *optimización robusta* renuncia a la distribución y exige factibilidad o buen coste para todo ξ de un *conjunto de incertidumbre* Ξ: $\min_x \max_{\xi\in\Xi} c(x,\xi)$. Para restricciones lineales con coeficientes en una caja, el problema robusto sigue siendo lineal.

Proposición 6.8 (Contrapartida robusta de una caja). *Sea $x \geq 0$ y $a_j \in [\bar{a}_j - \delta_j,\ \bar{a}_j + \delta_j]$ con $\delta_j \geq 0$. La restricción $a^{\mathsf{T}}x \leq b$ se cumple para todos los a de la caja si y solo si $(\bar{a}+\delta)^{\mathsf{T}}x \leq b$. Para x de signo libre, la condición es $\bar{a}^{\mathsf{T}}x + \delta^{\mathsf{T}}|x| \leq b$, que se linealiza con la proposición 1.6.*

La caja completa es muy conservadora: supone que todos los coeficientes toman su peor valor a la vez. Bertsimas y Sim [4] limitan a Γ el número de coeficientes que se desvían y controlan así el *precio de la robustez*: el coste que se paga por protegerse.

Observación 6.9 (La maldición del optimizador). Si se eligen la mejor de varias alternativas por su valor estimado, el valor estimado de la elegida sobrestima su valor real, aunque cada estimación sea insesgada: $\mathbb{E}[\max_i \hat{J}_i] \geq \max_i \mathbb{E}[\hat{J}_i]$ [36] (ejercicio 6.8). Por eso el beneficio que promete un optimizador sobre datos predichos debe validarse con datos nuevos, nunca con los usados para elegir.

6.4 Caso de estudio: plan de cerveza fresca con demanda predicha

Cuadro 6.1: Tecnologías del tema 6: predecir y optimizar bajo incertidumbre.

Herramienta	Para qué sirve	Licencia o tipo	Cuándo elegirla
scikit-learn	regresión, cuantiles (`GradientBoostingRegressor` con pérdida cuantílica), validación temporal	abierta (BSD-3)	modelos de predicción tabulares
SciPy / HiGHS	PL de SAA con escenarios	abierta (BSD-3; MIT)	programas estocásticos lineales de tamaño medio
CVXPY [12]	modelos robustos y con restricciones probabilísticas convexas	abierta (Apache 2.0)	contrapartidas robustas cónicas, CVaR
Pyomo	modelos estocásticos por escenarios y descomposición	abierta (BSD-3)	problemas grandes en dos o más etapas
PyTorch	capas de optimización diferenciables para aprender con la pérdida de decisión	abierta (BSD)	enfoques SPO o de aprendizaje orientado a la decisión

Bibliografía

[1] A. AhmadiTeshnizi, W. Gao y M. Udell. OptiMUS: Scalable optimization modeling with (MI)LP solvers and large language models. En *Proceedings of the 41st International Conference on Machine Learning*, PMLR 235:577–596, 2024. https://proceedings.mlr.press/v235/ahmaditeshnizi24a.html

[2] A. Ben-Tal, L. El Ghaoui y A. Nemirovski. *Robust Optimization*. Princeton University Press, 2009.

[3] D. Bertsimas y N. Kallus. From predictive to prescriptive analytics. *Management Science*, 66(3), 2020. Prepublicación de 2014. https://arxiv.org/abs/1402.5481

[4] D. Bertsimas y M. Sim. The price of robustness. *Operations Research*, 52(1):35–53, 2004.

[5] D. Bertsimas y J. N. Tsitsiklis. *Introduction to Linear Optimization*. Athena Scientific, 1997. Texto del curso 6.251J del MIT.

[6] J. R. Birge y F. Louveaux. *Introduction to Stochastic Programming*, 2.ª ed. Springer, 2011. Cap. 1, «Introduction and Examples».

[7] J. Blank y K. Deb. pymoo: Multi-objective optimization in Python. *IEEE Access*, 8, 2020. https://arxiv.org/abs/2002.04504

[8] S. Boyd y L. Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004. Cap. 4, «Convex optimization problems», y cap. 5, «Duality». Texto del curso EE364a de Stanford. https://web.stanford.edu/~boyd/cvxbook/

[9] S. Boyd y equipo docente de EE364a. Stochastic programming. Transparencias complementarias del curso EE364a: Convex Optimization I. Stanford University. https://web.stanford.edu/class/ee364a/lectures/stoch_prog.pdf

[10] S. P. Bradley, A. C. Hax y T. L. Magnanti. *Applied Mathematical Programming*. Addison-Wesley, 1977. Cap. 3, «Sensitivity Analysis», y cap. 9, «Integer Programming». Texto del curso 15.053 del MIT. https://web.mit.edu/15.053/www/

[11] K. Deb, A. Pratap, S. Agarwal y T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Transactions on Evolutionary Computation*, 6(2):182–197, 2002.

[12] S. Diamond y S. Boyd. CVXPY: A Python-embedded modeling language for convex optimization. *Journal of Machine Learning Research*, 17(83):1–5, 2016. https://www.jmlr.org/papers/v17/15-408.html

[13] Desarrolladores de CVXPY. CVXPY User Guide: «Disciplined Convex Programming» y «Advanced Features» (variables duales). https://www.cvxpy.org/tutorial/index.html

[14] A. N. Elmachtoub y P. Grigas. Smart «predict, then optimize». *Management Science*, 68(1):9–26, 2022. https://arxiv.org/abs/1710.08005

[15] R. Evans y J. Gao. DeepMind AI reduces Google data centre cooling bill by 40 %. Blog de Google DeepMind, 20 de julio de 2016. https://deepmind.google/discover/blog/deepmind-ai-reduces-google-data-centre-cooling-bill-by-40/

[16] P. I. Frazier. A tutorial on Bayesian optimization. Prepublicación, 2018. https://arxiv.org/abs/1807.02811

[17] R. Garnett. *Bayesian Optimization*. Cambridge University Press, 2023. Edición libre en línea. https://bayesoptbook.com/

[18] Google OR-Tools. CP-SAT Solver. Documentación oficial. https://developers.google.com/optimization/cp/cp_solver

[19] Google OR-Tools. The Job Shop Problem. Documentación oficial. https://developers.google.com/optimization/scheduling/job_shop

[20] Google OR-Tools. Vehicle Routing Problem with Time Windows. Documentación oficial. https://developers.google.com/optimization/routing/vrptw

[21] N. Hansen. The CMA evolution strategy: A tutorial. Prepublicación, 2016 (revisada en 2023). https://arxiv.org/abs/1604.00772

[22] HiGHS. High performance software for linear optimization. Documentación oficial; licencia MIT. https://highs.dev/

[23] C. Holland, J. Levis, R. Nuggehalli, B. Santilli y J. Winters. UPS optimizes delivery routes. *Interfaces*, 47(1):8–23, 2017.

[24] Q. Huangfu y J. A. J. Hall. Parallelizing the dual revised simplex method. *Mathematical Programming Computation*, 10(1):119–142, 2018.

[25] S. M. Johnson. Optimal two- and three-stage production schedules with setup times included. *Naval Research Logistics Quarterly*, 1(1):61–68, 1954.

[26] S. Kirkpatrick, C. D. Gelatt y M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.

[27] M. J. Kochenderfer y T. A. Wheeler. *Algorithms for Optimization*. MIT Press, 2019. Caps. 1 «Introduction», 7 «Direct Methods», 8 «Stochastic Methods», 9 «Population Methods», 12 «Multiobjective Optimization» y 17 «Optimization under Uncertainty».

[28] A. H. Land y A. G. Doig. An automatic method of solving discrete programming problems. *Econometrica*, 28(3):497–520, 1960.

[29] MIT OpenCourseWare. 6.251J / 15.081J: Introduction to Mathematical Programming. Massachusetts Institute of Technology, otoño de 2009. Clase 1 «Formulations», clases 9–12 (dualidad y sensibilidad) y 24–25 (optimización discreta), con lecturas de [5]. https://ocw.mit.edu/courses/6-251j-introduction-to-mathematical-programming-fall-2009/

[30] J. Orlin y E. Nasrabadi. 15.053: Optimization Methods in Management Science. Massachusetts Institute of Technology, primavera de 2013. Clases 2 «Formulations of linear and non-linear programs», 6 «Sensitivity analysis and shadow prices», 10–11 (programación entera), 12 «Branch and bound», 13 «Cutting planes» y 17 «Traveling salesman problem». https://ocw.mit.edu/courses/15-053-optimization-methods-in-management-science-spring-2013/

[31] J. Orlin y E. Nasrabadi. 15.053: Optimization Methods in Management Science. Hojas de problemas 4, 5 y 6. Massachusetts Institute of Technology, primavera de 2013. https://ocw.mit.edu/courses/15-053-optimization-methods-in-management-science-spring-2013/pages/assignments/

[32] J. Nocedal y S. J. Wright. *Numerical Optimization*, 2.ª ed. Springer, 2006. Cap. 12, teoría de la optimización con restricciones.

[33] A. Novikov, N. Vũ, M. Eisenberger, E. Dupont *et al.* AlphaEvolve: A coding agent for scientific and algorithmic discovery. Prepublicación, Google DeepMind, 2025. https://arxiv.org/abs/2506.13131

[34] S. Mitchell, A. Kean, A. Mason, M. O'Sullivan, A. Phillips y F. Peschiera. PuLP: documentación oficial (licencia MIT). https://coin-or.github.io/pulp/

[35] B. Romera-Paredes, M. Barekatain, A. Novikov *et al.* Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.

[36] J. E. Smith y R. L. Winkler. The optimizer's curse: Skepticism and postdecision surprise in decision analysis. *Management Science*, 52(3):311–322, 2006.

[37] M. M. Solomon. Algorithms for the vehicle routing and scheduling problems with time window constraints. *Operations Research*, 35(2):254–265, 1987.

[38] R. S. Sutton y A. G. Barto. *Reinforcement Learning: An Introduction*, 2.ª ed. MIT Press, 2018. http://incompleteideas.net/book/the-book-2nd.html

[39] A. J. Wood, B. F. Wollenberg y G. B. Sheblé. *Power Generation, Operation, and Control*, 3.ª ed. Wiley, 2014. Cap. 3, «Economic Dispatch of Thermal Units and Methods of Solution».

[40] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou y X. Chen. Large language models as optimizers. Prepublicación, 2023. https://arxiv.org/abs/2309.03409